

PROJECT REPORT

LegalEase

AI-Powered Legal Document Generator using Gemini

Project Type	AI / Generative AI Application
Technology	Gemini API, Web Application, Python/JavaScript
Academic Level	2nd BCA Project
Student Name	_____
Register Number	_____
College / Department	_____
Academic Year	2026-2027

Submitted as part of the Project Development Sessions

1. Brainstorming & Ideation Phase

Problem identified: Preparing legal documents can be time-consuming for users who do not know the correct structure, terminology, or formatting. Users may also struggle to identify what information is required in a document.

Proposed idea: LegalEase is a web-based AI assistant that accepts a user's document requirement and generates a structured draft using Google's Gemini API. The system is designed as an educational software project and does not replace a qualified legal professional.

The idea combines a simple form-based interface with generative AI. The user selects or describes the type of document, supplies relevant details, and receives a draft that can be reviewed, edited, copied, or exported.

Key features proposed during ideation:

- Simple user interface for entering document requirements.
- AI-powered draft generation using Gemini.
- Support for multiple common document categories such as letters, agreements, declarations, and notices.
- Structured output with headings, clauses, dates, parties, and other relevant fields.
- Input validation and clear error messages.
- Copy, download, or print generated content.
- A visible disclaimer that the generated output requires human review.

Expected outcome:

A functional prototype that demonstrates how generative AI can assist with structured document drafting while keeping the user in control of the final content.

2. Requirement Analysis Phase

2.1 Functional Requirements

- The system shall accept user details and document requirements.
- The system shall allow the user to choose a document type or describe the required document.
- The system shall send a properly formatted prompt to Gemini.
- The system shall display the generated document in a readable format.
- The user shall be able to edit or regenerate the draft.
- The system shall provide an option to copy or export the generated text.
- The system shall display errors when required inputs are missing or the API request fails.

2.2 Non-Functional Requirements

- Usability: The interface should be simple enough for a first-time user.
- Performance: The application should return a response within a reasonable time depending on API/network conditions.
- Security: API credentials must not be exposed in client-side source code.
- Reliability: API failures should be handled gracefully.
- Maintainability: Front-end, API, prompt, and utility modules should be separated.
- Privacy: Users should be informed not to enter unnecessary sensitive personal information.

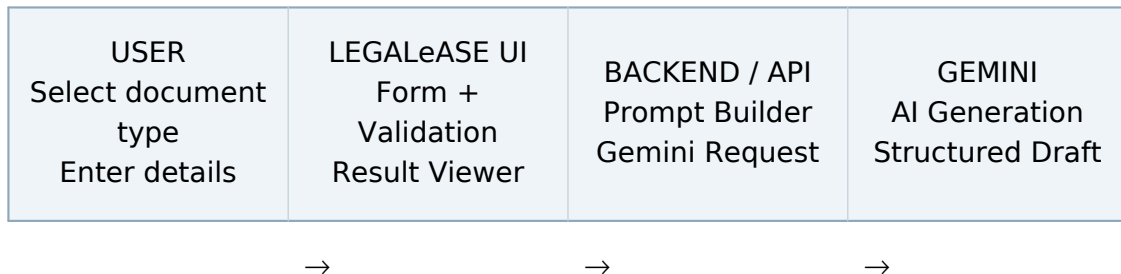
2.3 Hardware / Software Requirements

Category	Requirement
Processor	Any modern dual-core or better computer
RAM	4 GB minimum; 8 GB recommended
Storage	At least 1 GB free for project tools and dependencies
OS	Windows / Linux / macOS
Browser	Chrome, Edge, Firefox, or equivalent modern browser
Development	VS Code or equivalent IDE; Git/GitHub
AI Service	Google Gemini API

3. Project Design Phase

3.1 High-Level Architecture

The proposed architecture follows a simple three-layer flow: user interface → application/API layer → Gemini AI service. The generated result is returned to the interface for review and export.



3.2 Main Modules

- Input Module: collects document type, parties, purpose, dates, and other user-provided details.
- Validation Module: checks required fields and prevents empty or invalid submissions.
- Prompt Builder: converts user input into a controlled instruction for Gemini.
- Gemini Service Module: sends the request to the Gemini API and receives generated text.
- Output Module: formats the response into headings, clauses, and readable sections.
- Export Module: allows copy/download/print of the generated draft.

3.3 Suggested Data Flow

Input → Validate → Build Prompt → Call Gemini → Receive Response → Format → Display → User Review → Export

4. Project Planning Phase

The project is divided into manageable milestones so that each phase can be committed to GitHub separately, matching the required phase-wise submission structure.

Week / Milestone	Planned Work	Deliverable
1	Brainstorming & Ideation	Problem statement, idea, feature list
2	Requirement Analysis	Functional/non-functional requirements
3	Project Design	Architecture, modules, UI flow
4	Project Planning	Timeline, task allocation, repository structure
5-6	Development	Working UI + Gemini integration
7	Testing	Test cases, bug fixes, validation
8	Documentation	Final report, screenshots, README
9	Demonstration	Demo video and final presentation

4.1 GitHub Repository Structure

Recommended repository structure:

```
LegalEase/  
■■■ README.md  
■■■ docs/  
■ ■■■ 01_ideation.md  
■ ■■■ 02_requirements.md  
■ ■■■ 03_design.md  
■ ■■■ 04_planning.md  
■ ■■■ 05_development.md  
■ ■■■ 06_testing.md  
■ ■■■ 07_documentation.md  
■ ■■■ 08_demonstration.md  
■■■ frontend/  
■■■ backend/  
■■■ prompts/  
■■■ tests/  
■■■ screenshots/
```

Repository visibility: Public repository, as required for submission. Each phase should be committed and documented separately.

5. Project Development Phase

5.1 Technology Stack

- Frontend: HTML, CSS, JavaScript (or React if selected by the team).
- Backend: Python Flask/FastAPI or Node.js/Express.
- AI: Google Gemini API.
- Version Control: Git and GitHub.
- Development Environment: VS Code and a modern web browser.

Google's current Gemini documentation describes the Gemini API as supporting text generation and multimodal inputs, and provides SDK/API approaches for application integration.

`url` Gemini API documentation `https://ai.google.dev/gemini-api/docs`

5.2 Core Processing Logic

A simplified implementation flow is shown below:

1. Read user inputs
2. Validate mandatory fields
3. Create a structured prompt
4. Send the prompt to the Gemini API
5. Receive generated content
6. Apply formatting and safety checks
7. Display the draft
8. Allow the user to edit and export

5.3 Example Prompt Pattern

You are a legal-document drafting assistant for an educational application. Generate a clear, structured draft based only on the user's supplied information. Use headings and numbered clauses where appropriate. Do not invent missing facts. Clearly state that the output is a draft requiring human review.

The application should keep the Gemini API key on the server side or in a protected environment variable rather than exposing it in browser code. Google's documentation notes that Gemini API requests require an API key for authentication.

`cite` `turn0search3` `turn0search4`

6. Project Testing Phase

Testing verifies that the system accepts valid information, rejects incomplete inputs, communicates with Gemini correctly, and presents a usable result.

Test ID	Test Case	Expected Result	Status
TC01	Open application	Home page loads correctly	Pass / To Test
TC02	Submit empty form	Validation message displayed	Pass / To Test
TC03	Generate a document with valid input	App-generated draft displayed	Pass / To Test
TC04	Gemini/API failure	Friendly error message displayed	Pass / To Test
TC05	Regenerate document	New draft is generated	Pass / To Test
TC06	Copy output	Generated text copied successfully	Pass / To Test
TC07	Download/print	Output can be saved or printed	Pass / To Test
TC08	Mobile browser	Layout remains usable	Pass / To Test

Testing checklist

- Verify required fields before every AI request.
- Verify that API errors do not crash the application.
- Check that generated output is displayed without broken formatting.
- Test different document types and different lengths of input.
- Verify that the user can edit the generated draft before saving it.
- Confirm that sensitive information is not unnecessarily logged.

7. Project Documentation Phase

7.1 Project Abstract

LegalEase is an AI-powered legal document drafting application designed to simplify the creation of structured document drafts. The application uses a web interface to collect user requirements and the Gemini API to generate an organized draft. The project demonstrates the practical use of generative AI, prompt engineering, API integration, input validation, and user-centered interface design.

7.2 Objectives

- 1 To develop a simple application for generating structured legal-document drafts.
- 2 To integrate a generative AI service through an API.
- 3 To reduce repetitive manual formatting work.
- 4 To demonstrate prompt engineering and controlled AI output.
- 5 To provide an editable result that keeps the user responsible for final review.

7.3 Advantages

- Reduces time spent creating a first draft.
- Provides consistent document structure.
- Easy-to-use interface.
- Can support multiple document categories.
- Demonstrates real-world use of Generative AI.

7.4 Limitations

- Generated content may contain errors or omissions.
- The quality of output depends on the information supplied by the user and the AI model.
- Internet access and API availability are required for cloud-based generation.
- The generated document should not be treated as professional legal advice.

7.5 Future Enhancements

- User accounts and secure document history.
- PDF/DOCX export with professional formatting.
- Multiple-language support.
- Document comparison and version history.
- Optional retrieval of approved reference material for controlled drafting.
- Admin dashboard for project usage analytics.

8. Project Demonstration Phase

The demonstration video should show the complete working flow from opening the application to generating and reviewing a document.

Recommended demonstration sequence

- 1 Introduce the project title and problem statement.
- 2 Open the LegalEase application.
- 3 Show the available document types or input form.
- 4 Enter sample, non-sensitive information.
- 5 Submit the request and show Gemini-generated output.
- 6 Explain the structure of the generated draft.
- 7 Edit a small part of the output to demonstrate user control.
- 8 Use copy/download/print functionality if implemented.
- 9 Briefly explain the technology stack and architecture.
- 10 End with limitations and future enhancements.

Demo video submission checklist

- Record the complete project demonstration.
- Upload the video to Google Drive.
- Set sharing to Public/Anyone with the link can view.
- Add the Drive link to the final submission document or submission form.
- Ensure the video clearly shows the application working.

Submission links to fill in

Item	Link / Details
GitHub Public Repository	_____
Google Drive Demo Video	_____
Live Project URL (if available)	_____

Appendix A - Sample User Flow

User opens LegalEase → selects document category → enters details → validates input → submits → Gemini generates draft → result is displayed → user reviews/edits → user exports the final draft.

Appendix B - Sample README Content

LegalEase - AI-Powered Legal Document Generator using Gemini

LegalEase is a student project demonstrating how a Generative AI API can be integrated into a web application for structured document drafting.

Features

- Document requirement input
- Gemini-powered generation
- Structured output
- Validation and error handling
- Editable result
- Export/copy support

Setup overview

1. Clone the public GitHub repository.
2. Install project dependencies.
3. Configure the Gemini API key as an environment variable.
4. Start the backend and frontend.
5. Open the application in a browser.

Important: Do not commit API keys or other secrets to GitHub.

Appendix C - Academic Declaration

This project is submitted as an academic software project. The application is intended to demonstrate programming, API integration, prompt design, testing, and documentation skills. Generated legal content is a draft for demonstration purposes and should be reviewed by a qualified professional before any real-world use.